

supervisor	
signature	

※ All your answers should be written in English. Ensure that your handwriting is clearly readable.

The diagram illustrates the difference between single-threaded and multithreaded processes and how they relate to fine-grain and coarse-grain parallelism.

Single-threaded process: A sequence of steps (a), (b), (c), (d), and (e) executed sequentially by a single thread.

Multithreaded process: The same steps are executed by multiple threads. Steps (a), (b), and (c) are shared among threads, while steps (d) and (e) are maintained by each thread. This is labeled as "shared among threads" and "maintained by each thread".

Parallelism: The diagram shows a timeline where fine-grain parallelism (small orange rectangles) is contrasted with coarse-grain parallelism (large orange rectangles). A white rectangle represents idle time (f), and a black rectangle represents busy time (g).

This is a typical view of fine-grain and coarse-grain parallelism.

A white rectangle in this diagram represents (f).
A black rectangle in this diagram represents (g).

4.(38 points) (a) Consider a multi-threaded Java program (**SyncMethodTest.java**). The main function creates two threads, **Incrementer** and **Decrementer**, which run concurrently. The program uses a shared data **Count**, initialized to 0. The **Incrementer** thread increments **Count** in a loop for 10001234 iterations and the **Decrementer** thread decrements **Count** in a loop for 10000000 iterations. Both threads execute the operations concurrently. The main thread waits for both the **Incrementer** and **Decrementer** threads to complete the operations. After both threads have terminated, the main thread wakes up and prints the final value of **Count** as a result, which should be 1234. Fill in the empty boxes below with appropriate Java code using the **synchronized** keyword to ensure thread safety and prevent race conditions. (Note: 'increment' means adding 1; 'decrement' means subtracting 1)

(b) Consider a multi-threaded Java program (**ex2.java**) that computes and prints the sum of elements in a given array using a divide-and-conquer approach. In this program, each thread recursively creates two new threads to compute the sums of the left half and the right half of the array, respectively. **SEQUENTIAL_CUTOFF** needs to be used to reduce the depth of recursion and limit thread creation. Fill in the empty boxes with appropriate Java code.

-----< source code >-----

(a) **SyncMethodTest.java**: using the **synchronized** keyword

```

public class SyncMethodTest {
    public static void main(String[] args) {
        int inc_num = 10001234;
        int dec_num = 10000000;

        [ ]

    }
}

class incrementer [ ] {
    [ ]
}

class decrementer [ ] {
    [ ]
}

[ ]
        
```

(b) **ex2.java**: summation using divide-and-conquer approach

```

class SumThread [ ] {
    int lo; int hi;
    int[] arr;
    int ans = 0; // for communicating result
    private static int SEQUENTIAL_CUTOFF=100;
    SumThread(int[] a,int l,int h) { lo=l; hi=h; arr=a; }

    [ ]
}

public class ex2 {
    private static int NUM_END=1000;
    public static void main(String[] args) {
        int[] int_arr = new int [NUM_END];
        int i,s;
        for (i=0;i<NUM_END;i++) int_arr[i]=i+1;
        s=sum(int_arr);
        System.out.println(s);
    }

    static int sum(int[] arr) {
        [ ]
    }
}
        
```